LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

Level 7 – Cyber Security and Artificial Intelligence

Assessment Type

60% Individual Coursework

Semester

2025 Autumn

Credit: 20 Semester Long Module

Student Name: Abhiyan Shrestha

London Met ID: 24059497

College ID: np01ms7s250025

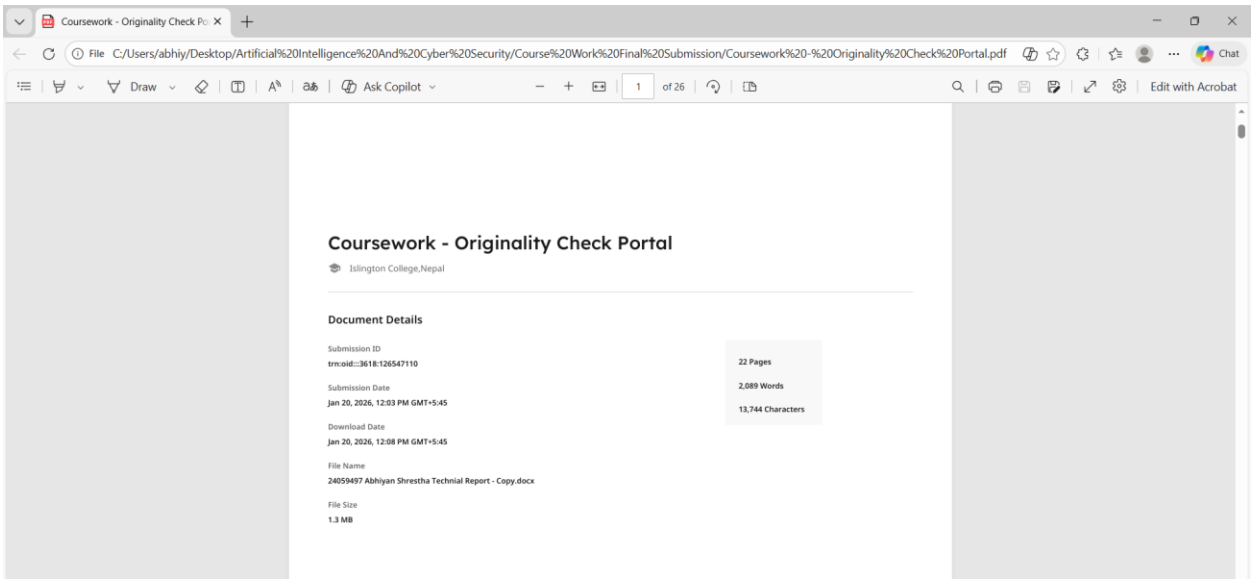
Assignment Due Date: Sunday, January 25, 2026

Assignment Submission Date: Sunday, January 25, 2026

Submitted To: Suraj Neupane

Word Count (Where Required): 2089

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.



Coursework - Originality Check Portal

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:126547110

Submission Date
Jan 20, 2026, 12:03 PM GMT+5:45

Download Date
Jan 20, 2026, 12:08 PM GMT+5:45

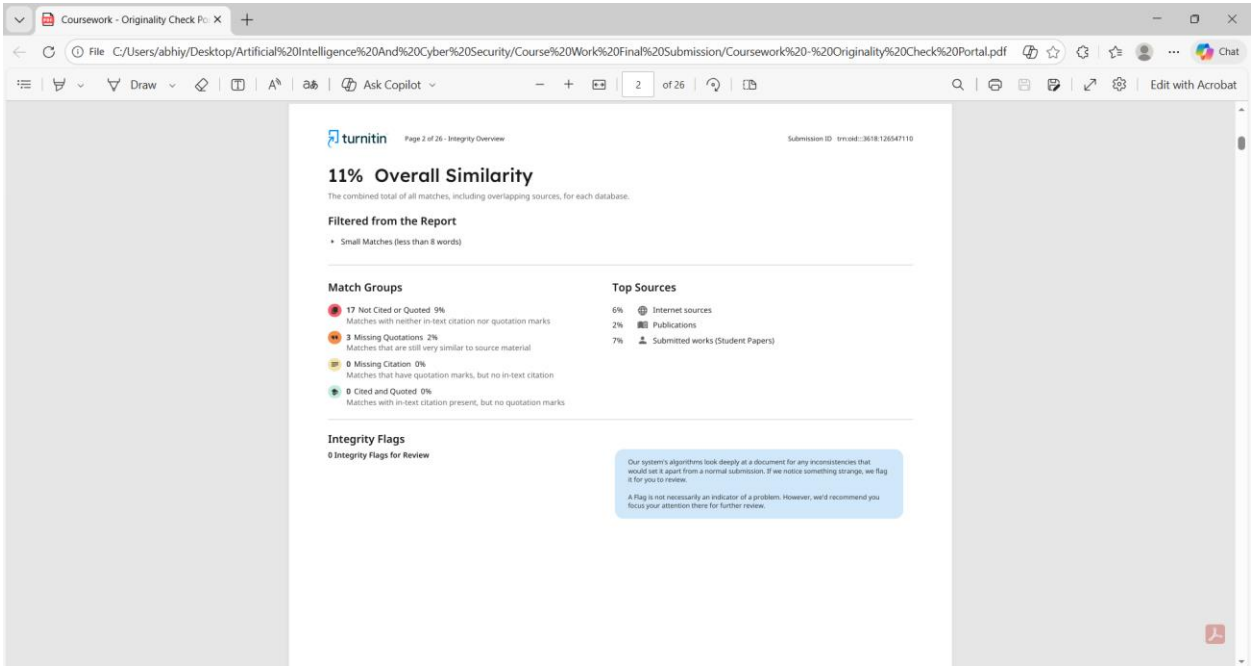
File Name
24059497 Abhiyan Shrestha Technial Report - Copy.docx

File Size
1.3 MB

22 Pages

2,089 Words

13,744 Characters



turnitin Page 2 of 26 - Integrity Overview Submission ID trn:oid:::3618:126547110

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Small Matches (less than 8 words)

Match Groups

- 17 Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- 3 Missing Quotations 2%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

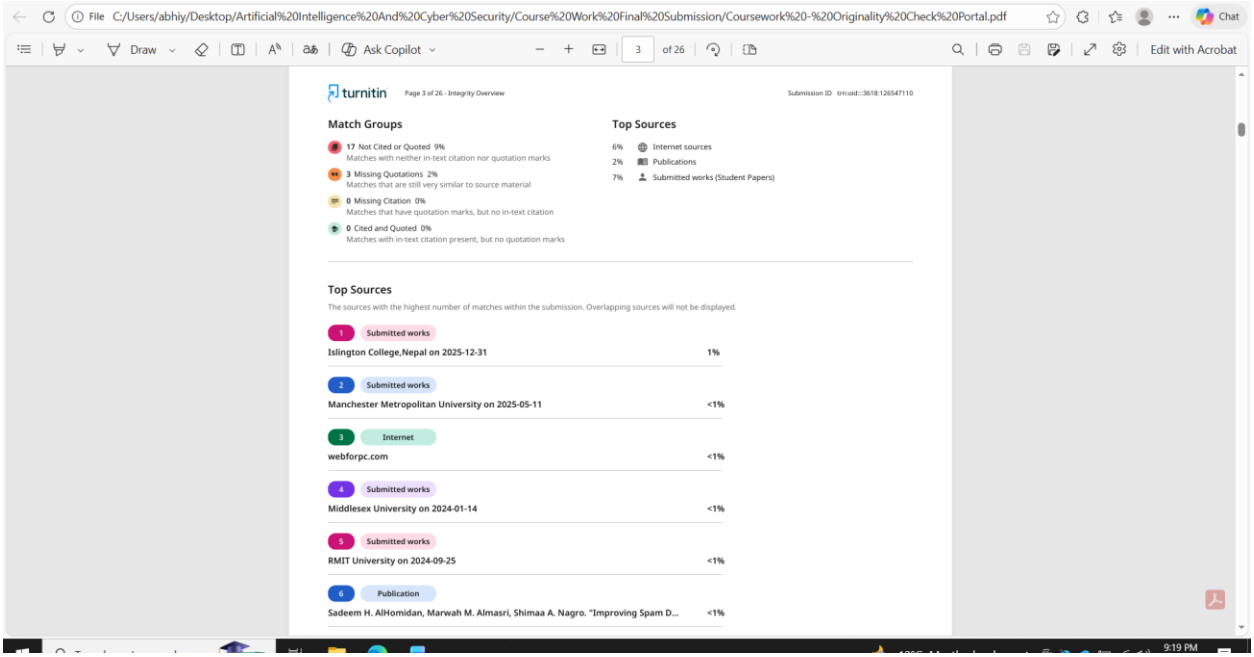
6% Internet sources
2% Publications
7% Submitted works (Student Papers)

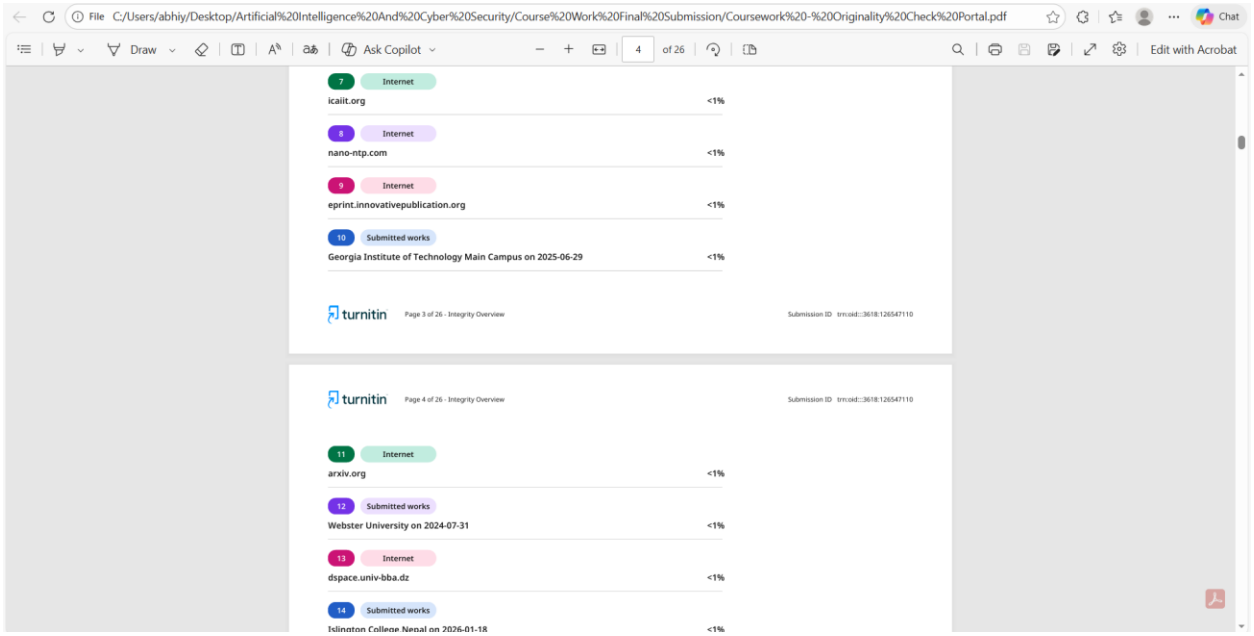
Integrity Flags

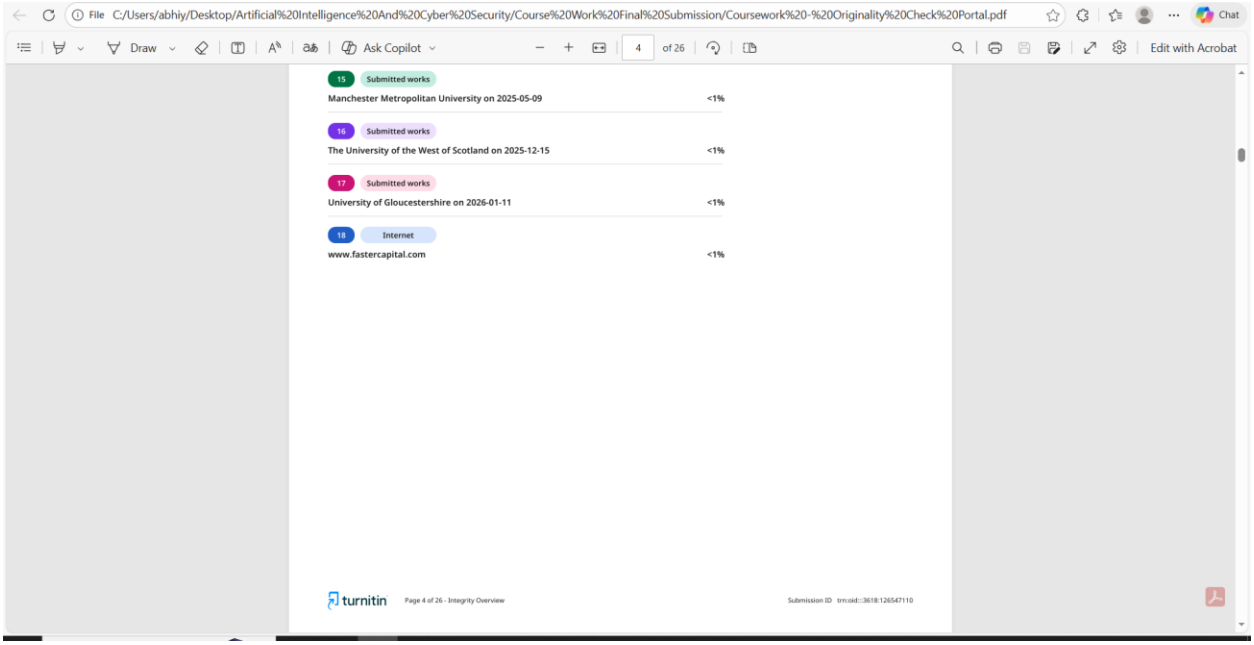
0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.


File C:/Users/abhiy/Desktop/Artificial%20Intelligence%20And%20Cyber%20Security/Course%20Work%20Final%20Submission/Coursework%20-%20Originality%20Check%20Portal.pdf
Draw
Ask Copilot
3
of 26
Edit with Acrobat
turnitin
Page 3 of 26 - Integrity Overview
Submission ID trn:oid:::3618:126547110
Match Groups
17 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks
3 Missing Quotations 2%
Matches that are still very similar to source material
0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation
0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks
Top Sources
6% Internet sources
2% Publications
7% Submitted works (Student Papers)
Top Sources
The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.
1 Submitted works
Islington College,Nepal on 2025-12-31 1%
2 Submitted works
Manchester Metropolitan University on 2025-05-11 <1%
3 Internet
webforpc.com <1%
4 Submitted works
Middlesex University on 2024-01-14 <1%
5 Submitted works
RMIT University on 2024-09-25 <1%
6 Publication
Sadeem H. AlHomidan, Marwah M. Almasri, Shimaa A. Nagro. "Improving Spam D... <1%
9:19 PM


File C:/Users/abhiy/Desktop/Artificial%20Intelligence%20And%20Cyber%20Security/Course%20Work%20Final%20Submission/Coursework%20-%20Originality%20Check%20Portal.pdf
Draw
Ask Copilot
4
of 26
Edit with Acrobat
7 Internet
icaiit.org <1%
8 Internet
nano-ntp.com <1%
9 Internet
eprint.innovativepublication.org <1%
10 Submitted works
Georgia Institute of Technology Main Campus on 2025-06-29 <1%
turnitin
Page 3 of 26 - Integrity Overview
Submission ID trn:oid:::3618:126547110
turnitin
Page 4 of 26 - Integrity Overview
Submission ID trn:oid:::3618:126547110
11 Internet
arxiv.org <1%
12 Submitted works
Webster University on 2024-07-31 <1%
13 Internet
dspace.univ-bba.dz <1%
14 Submitted works
Islington College Nepal on 2026-01-18 <1%


File C:/Users/abhiy/Desktop/Artificial%20Intelligence%20And%20Cyber%20Security/Course%20Work%20Final%20Submission/Coursework%20-%20Originality%20Check%20Portal.pdf
Draw
Ask Copilot
4
of 26
Edit with Acrobat
15 Submitted works
Manchester Metropolitan University on 2025-05-09 <1%
16 Submitted works
The University of the West of Scotland on 2025-12-15 <1%
17 Submitted works
University of Gloucestershire on 2026-01-11 <1%
18 Internet
www.fastercapital.com <1%
turnitin
Page 4 of 26 - Integrity Overview
Submission ID trn:oid:::3618:126547110

Executive Summary

Phishing attacks remain one of the most prevalent and damaging cyber threats, exploiting human trust rather than technical vulnerabilities. Email continues to be the primary attack vector, with adversaries crafting increasingly sophisticated phishing messages that bypass traditional rule-based spam filters. These attacks can lead to credential theft, financial fraud, identity theft, and large-scale data breaches. As organizations and individuals rely heavily on cloud-based email services such as Gmail, there is a critical need for intelligent, adaptive, and real-time phishing detection mechanisms.

This technical report presents the design, development, and evaluation of an **AI-Based Phishing Email Detection System with Gmail Integration**. The system leverages machine learning (ML) and natural language processing (NLP) techniques to automatically classify emails as phishing or legitimate. A Logistic Regression model trained on a Kaggle phishing email dataset is integrated with the Gmail API using OAuth 2.0 authentication. The system continuously scans unread emails in a user's Gmail inbox, analyzes their content, and automatically moves detected phishing emails to the Trash. A desktop graphical user interface (GUI) built with Tkinter provides transparency, usability, and real-time status updates.

The implementation demonstrates that classical machine learning models, when combined with effective text preprocessing and TF-IDF feature extraction, can achieve high accuracy while remaining lightweight and explainable. Experimental results show strong classification performance on held-out test data, and live testing confirms the system's ability to operate safely in a real-world Gmail environment. The report also discusses limitations, ethical considerations, and future improvements such as deep learning models, attachment and URL reputation analysis, and enterprise-scale deployment.

Table of Contents

Introduction

Introduction to the Topic

Electronic mail remains one of the most widely used communication mechanisms in both personal and professional environments due to its convenience, low cost, and global accessibility (Fette, 2007). However, this widespread adoption has also made email one of the most attractive attack vectors for cybercriminals. Among various email-based threats, phishing attacks have emerged as one of the most persistent and damaging forms of cybercrime. Phishing involves the use of deceptive emails that impersonate trusted individuals, organizations, (Fette, 2007) or services to manipulate recipients into revealing sensitive information such as login credentials, financial details, or personal data. In many cases, phishing attacks serve as the initial entry point for more complex cyber incidents, including ransomware infections, identity theft, and large-scale data breaches.

The growing sophistication of phishing attacks has significantly reduced the effectiveness of traditional email security mechanisms. Conventional spam filters and rule-based detection systems rely on predefined signatures, blacklists, or heuristic rules that struggle to keep pace with evolving attack techniques. Modern phishing campaigns often employ carefully crafted language, legitimate-looking sender addresses, domain spoofing, and personalized content to evade detection (Fette, 2007) . Attackers may also leverage URL shorteners, embedded HTML content, or socially engineered urgency cues to exploit human psychology rather than technical vulnerabilities. As a result, phishing emails frequently bypass standard filters and reach end users, placing them at considerable risk.

The increasing reliance on cloud-based email services, particularly platforms such as Gmail, has further amplified the importance of robust phishing detection mechanisms. Gmail serves billions of users worldwide and is deeply integrated into both individual workflows and organizational infrastructures (Fette, 2007). Although Gmail incorporates built-in security features, no system can guarantee complete protection against all phishing variants. Consequently, there is a growing need for supplementary, user-controlled security solutions that provide an additional layer of intelligent defense. Such solutions should be capable of adapting to new threats, operating in real time, and integrating seamlessly with existing email platforms without compromising user privacy (Fette, 2007).

Artificial intelligence and machine learning have emerged as powerful tools in addressing cybersecurity challenges, including phishing detection. Machine learning models can be trained on large volumes of historical email data to learn patterns that distinguish phishing emails from legitimate communications. Unlike static rule-based systems, machine learning approaches are inherently adaptive and can generalize from past examples to detect previously unseen attacks. When combined with natural language processing techniques, these models can analyze the semantic and contextual characteristics of email content, enabling more accurate and nuanced classification decisions.

Natural language processing plays a critical role in phishing detection by transforming unstructured email text into structured representations suitable for machine learning. Techniques such as tokenization, stopword removal, lemmatization, and feature extraction allow systems to focus on meaningful linguistic patterns while reducing noise. Feature representation methods such as Term Frequency–Inverse Document Frequency (TF-IDF) quantify the importance of words within emails and across datasets, making them particularly effective for text-based classification tasks. Lightweight yet robust classifiers, such as Logistic Regression, have demonstrated strong performance in phishing detection while maintaining computational efficiency and interpretability.

In addition to accurate detection, practical deployment considerations are essential for real-world phishing mitigation systems. Secure integration with email services must adhere to strict authentication and authorization standards to protect user data. OAuth 2.0 has become the industry standard for granting limited, consent-based access to cloud services without exposing user credentials. Furthermore, effective phishing mitigation strategies should prioritize safety and reversibility, ensuring that legitimate emails are not permanently lost due to misclassification. User interfaces that provide visibility into system behavior are also crucial for trust, transparency, and usability.

This project situates itself within this broader context by focusing on the design and implementation of an AI-based phishing email detection system integrated with Gmail. By combining machine learning, natural language processing, and secure API-based automation, the project aims to demonstrate how intelligent systems can enhance email security in a practical, ethical, and user-centric manner. The work reflects current trends in cybersecurity research and provides a foundation for future advancements in AI-driven email protection.

Problem Statement

Although Gmail and other major email providers deploy advanced security filters, phishing emails still reach end users. The key problems motivating this project are:

- High Risk of Data Breaches: Users may unknowingly click malicious links or share credentials.

- Limitations of Rule-Based Filters: Static rules struggle to detect novel or context-aware phishing attacks.

- Lack of User-Centric Transparency: Users often do not understand why an email was flagged or missed.

- Delayed or Manual Response: Many users only act after interacting with a phishing email, increasing risk.

These challenges highlight the need for an additional, user-controlled layer of intelligent phishing detection that operates in real time and integrates directly with the user's email inbox.

Project as a Solution

This project addresses the above problems by developing a standalone, AI-driven phishing detection system integrated with Gmail. The system:

- Uses supervised machine learning trained on a labeled phishing dataset.
- Applies NLP-based preprocessing and TF-IDF feature extraction.
- Integrates securely with Gmail using OAuth 2.0.
- Automatically scans unread emails in real time.
- Takes safe mitigation action by moving phishing emails to Trash.
- Provides a desktop GUI for monitoring and transparency.

The solution is designed to be modular, explainable, and extensible, making it suitable for academic, personal, or prototype enterprise use.

Aim and Objectives

Aim

The primary aim of this project is:

To design and implement an AI-based system capable of detecting and mitigating phishing emails in Gmail in real time, thereby enhancing email security and reducing user exposure to social engineering attacks.

Objectives

To achieve this aim, the following objectives were defined:

- Collect and preprocess a phishing email dataset from Kaggle.
- Apply NLP techniques such as tokenization, stopword removal, and lemmatization.
- Extract discriminative textual features using TF-IDF.
- Train and evaluate a supervised machine learning model for phishing classification.
- Integrate the trained model with Gmail using OAuth 2.0 authentication.
- Implement automated actions for detected phishing emails.

- Develop a desktop GUI for real-time monitoring and usability.

- Test and evaluate the system using both offline metrics and live Gmail data.

Technical Background

Phishing Email Characteristics

Phishing emails often exhibit linguistic and structural patterns such as:

- Urgency cues (e.g., “urgent”, “account suspended”).
- Requests for sensitive information.
- Presence of URLs or shortened links.
- Brand impersonation and spoofed sender addresses.

These patterns make text-based analysis a suitable approach for detection.

Natural Language Processing (NLP)

NLP enables machines to process and analyze human language. In this system, NLP techniques are used to normalize and clean email text to reduce noise and improve model performance. The preprocessing pipeline includes:

- Lowercasing text.
- Removing HTML tags.
- Replacing URLs with a generic token.
- Removing special characters and punctuation.
- Stopword removal using NLTK.
- Lemmatization using WordNet.

This process ensures that semantically similar words are treated consistently and irrelevant tokens are removed.

Feature Extraction using TF-IDF

The Term Frequency–Inverse Document Frequency (TF-IDF) vectorizer converts cleaned text into numerical feature vectors. TF-IDF assigns higher weights to words that are frequent in a specific document but rare across the corpus, making it effective for distinguishing phishing-related terms.

Key configuration:

- Maximum features: 5,000
- Unigrams (single words)

Machine Learning Model

A Logistic Regression classifier, wrapped in a One-vs-Rest strategy, is used for binary classification (phishing vs safe). Logistic Regression was chosen due to:

- High performance on text classification tasks.
- Interpretability and explainability.
- Low computational overhead.
- Suitability for real-time applications.

Evaluation Metrics

The system is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

Accuracy is reported during training, while qualitative evaluation is performed during live Gmail testing.

Development and Implementation

System Architecture Overview

The system architecture consists of the following major components:

- Data Layer (Kaggle dataset)
- Preprocessing Module
- Machine Learning Model
- Gmail API Integration Module
- Action Module
- GUI Dashboard

Emails flow from the Gmail inbox through the API, are analyzed by the ML model, and appropriate actions are taken.

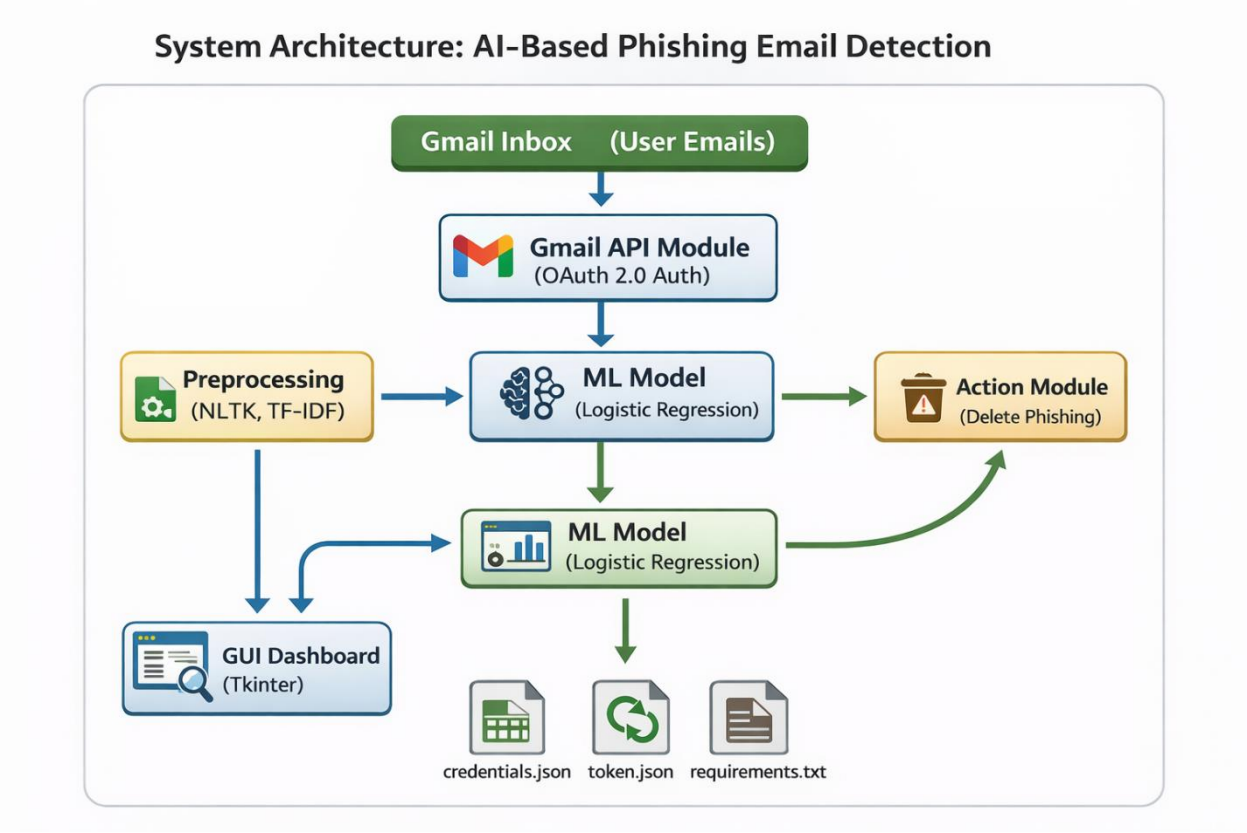


Fig: System Architecture

Dataset and Preprocessing Implementation

The Kaggle phishing email dataset is loaded using Pandas. The preprocessing module (preprocess.py) automatically detects text and label columns, cleans the email body, normalizes labels, and saves the processed dataset.

Automatic label mapping ensures compatibility with different dataset formats.

Model Training and Persistence

The training script (train_model.py) performs:

- Train-test split (80/20).
- TF-IDF vectorization.
- Logistic Regression training.
- Accuracy evaluation.

The trained model and vectorizer are serialized using Pickle and stored for reuse.

Prediction Module

The prediction module (predict.py) loads the saved model and vectorizer and exposes a predict_email() function for real-time inference.

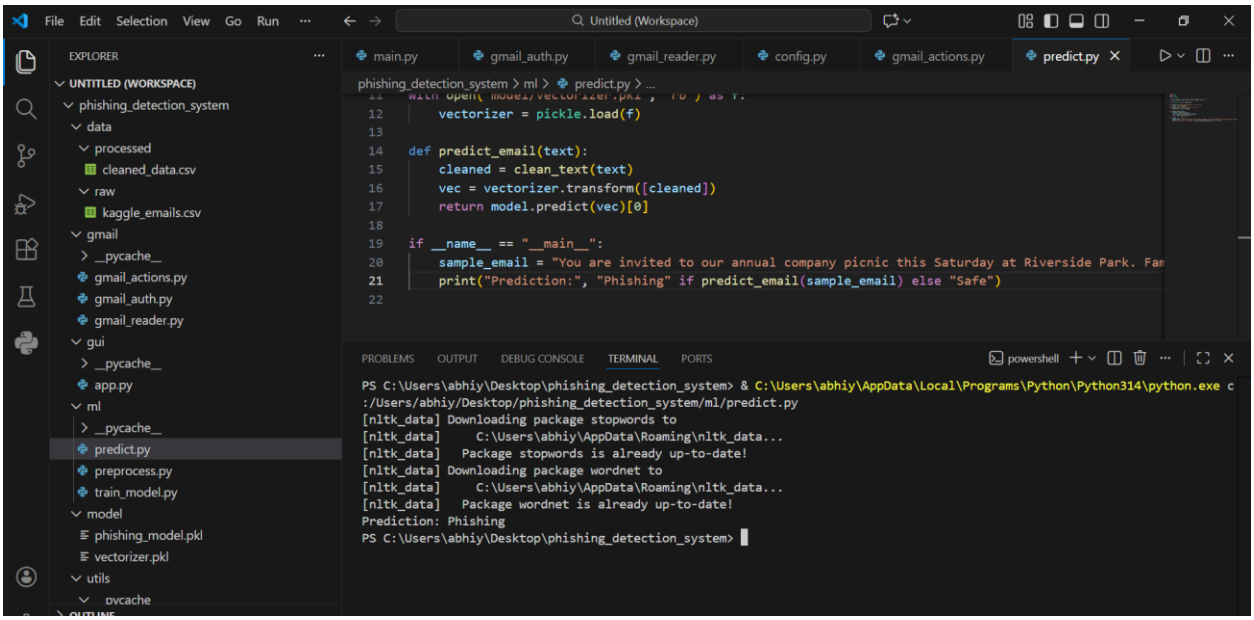


Fig: Predict.py Code

Gmail API Integration

Secure Gmail access is implemented using OAuth 2.0. The gmail_auth.py module handles token management and scope validation. The system uses read-only and modifies scopes to safely read and trash emails.

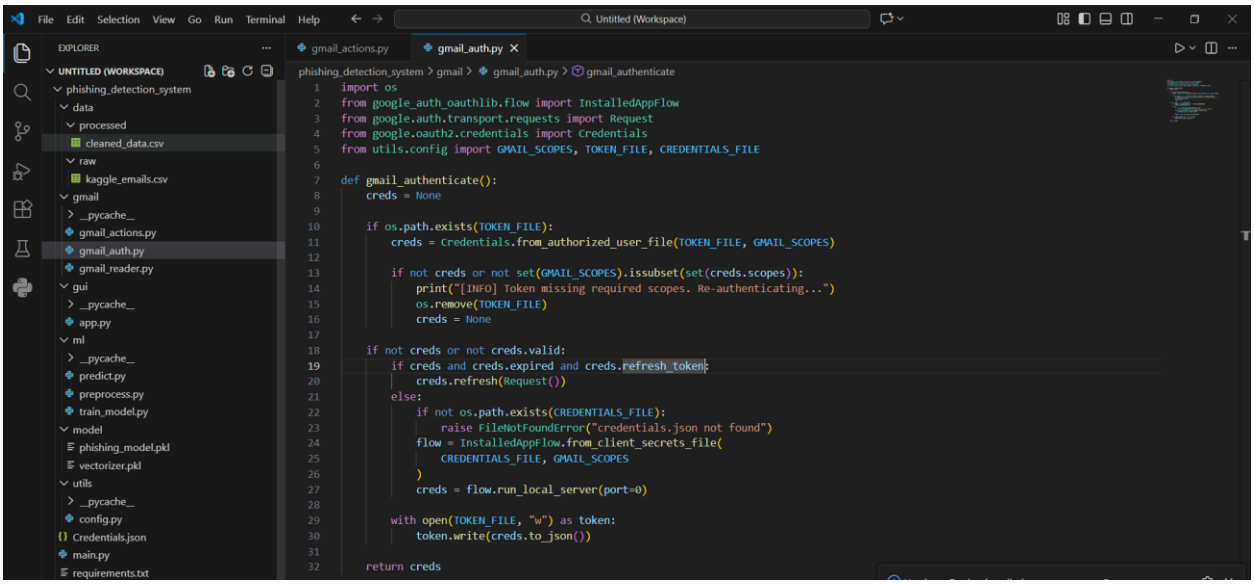


Fig: gmail_auth.py Code

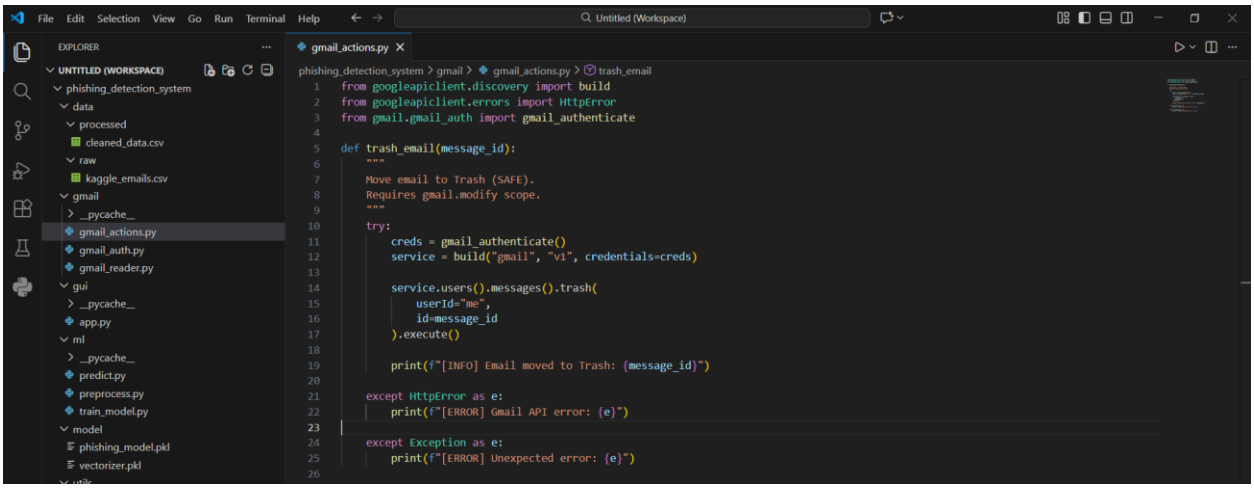


Fig: gmail_actions.py Code

Unread emails are fetched, decoded from Base64, and parsed for subject, sender, and body.

Action Module

Detected phishing emails are moved to Trash using the Gmail API. This approach is safer than permanent deletion and allows recovery if false positives occur.

GUI Implementation

A Tkinter-based GUI provides:

- Real-time scanning status.
- Display of sender, subject, and classification result.
- Background scanning using threading.

The GUI improves usability and transparency.

Testing and Evaluation

Offline Evaluation

The trained model achieves high accuracy on the test dataset, demonstrating effective discrimination between phishing and legitimate emails.

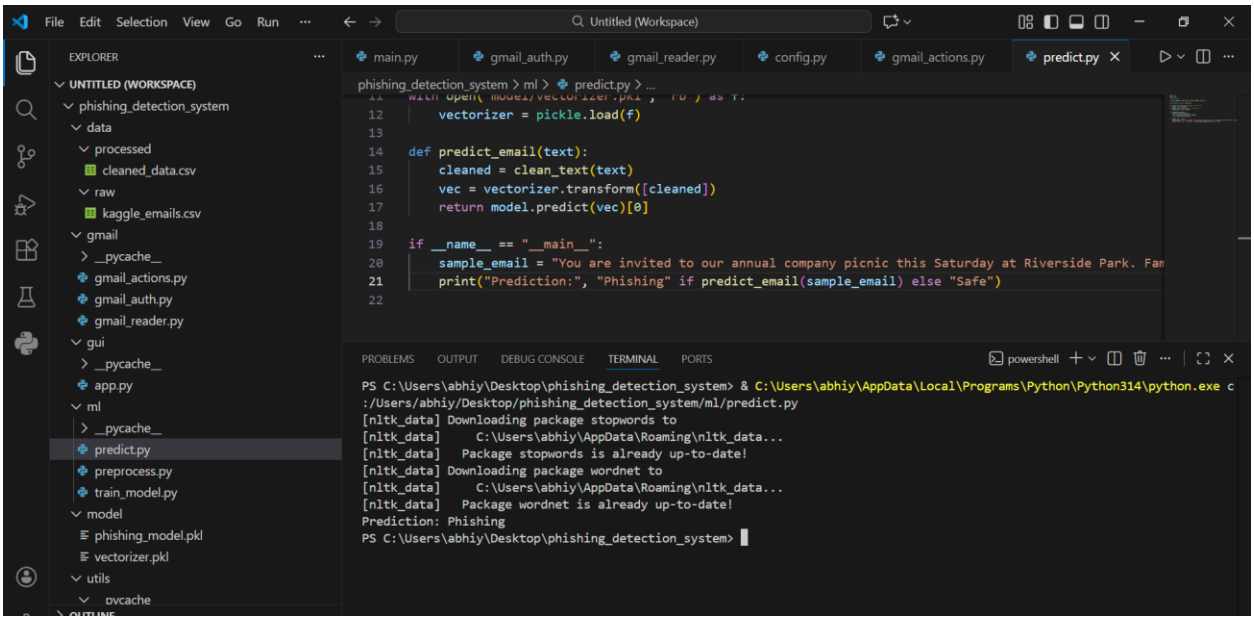


Fig: Predict.py Code offline testing

Live Gmail Testing

During live testing:

- Phishing-like emails were correctly flagged and trashed.
- Legitimate emails were preserved.
- The system operated without disrupting Gmail functionality.

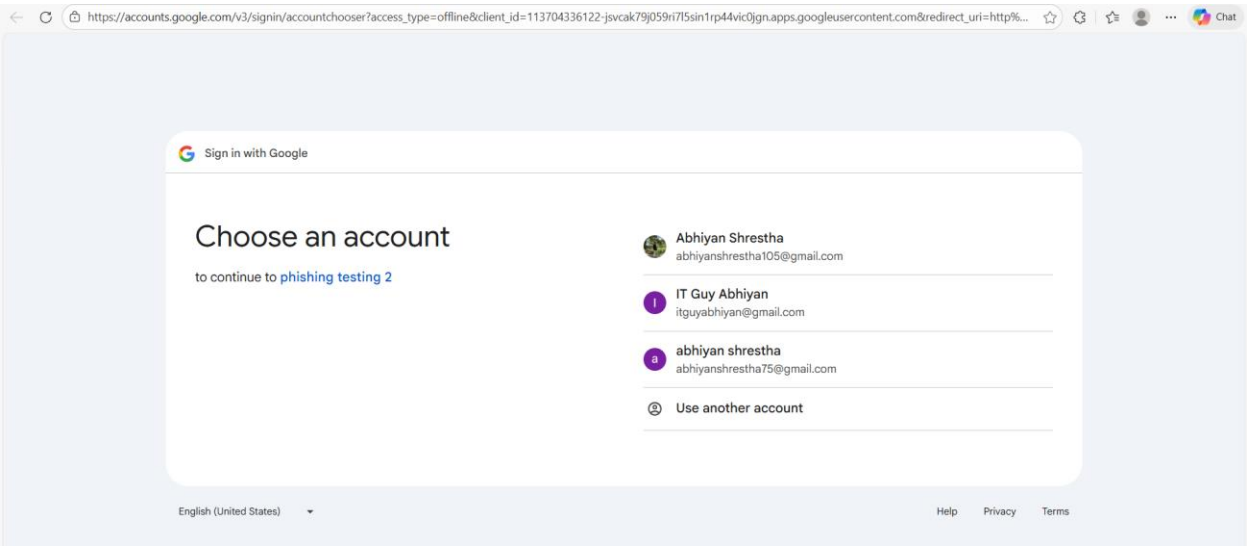


Fig: Choosing the email which i want to connect.

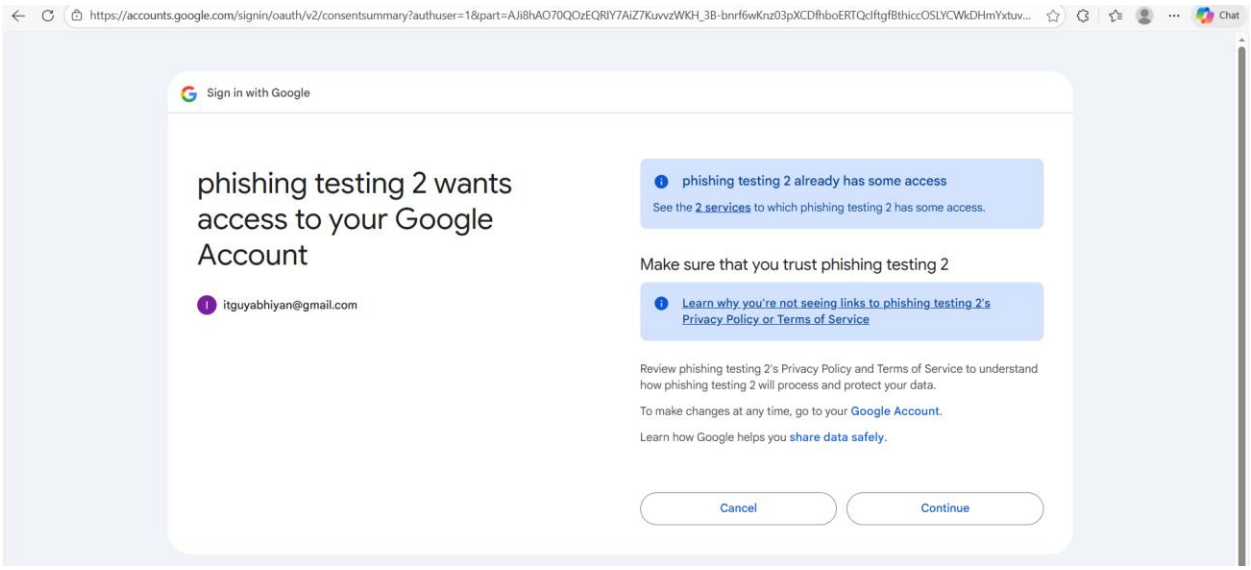


Fig: Asking for permission to access and modify the mail

Fig: Connection is Successful

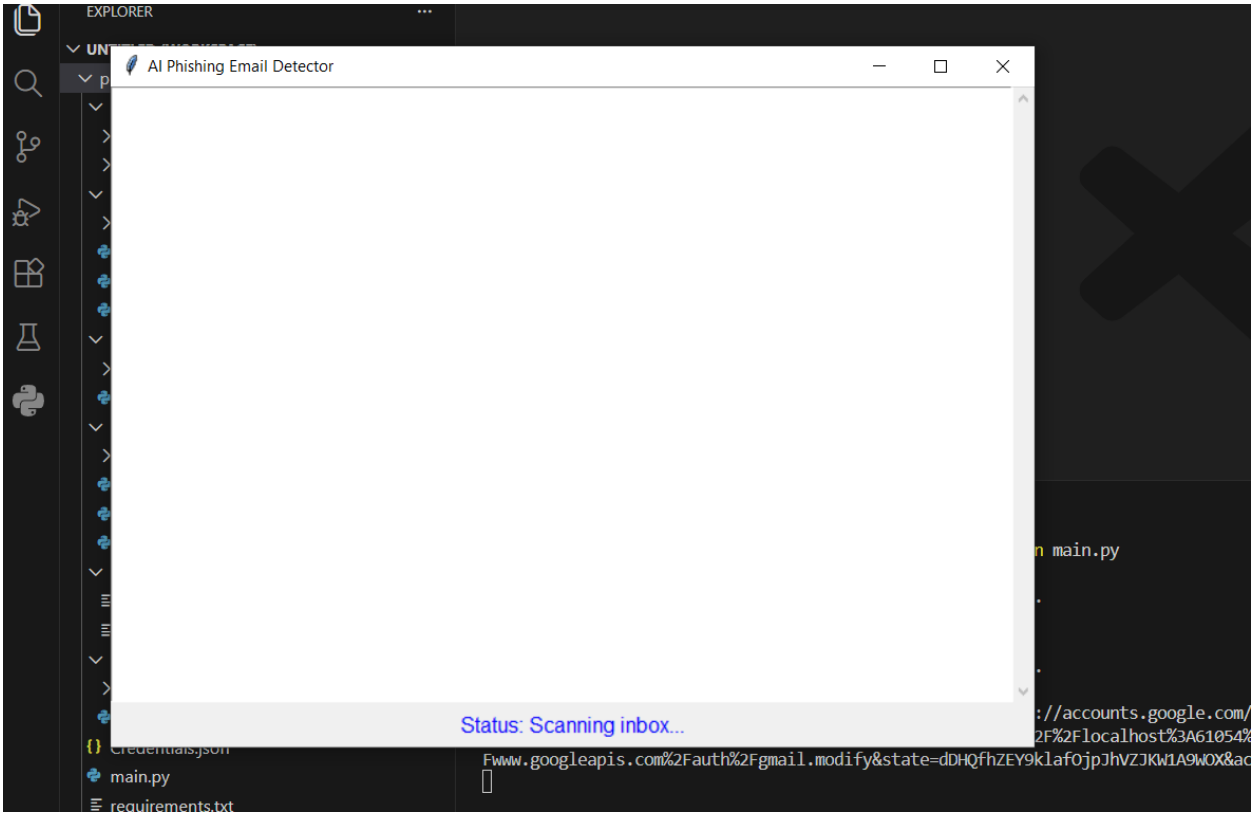


Fig: The System is Scanning emails

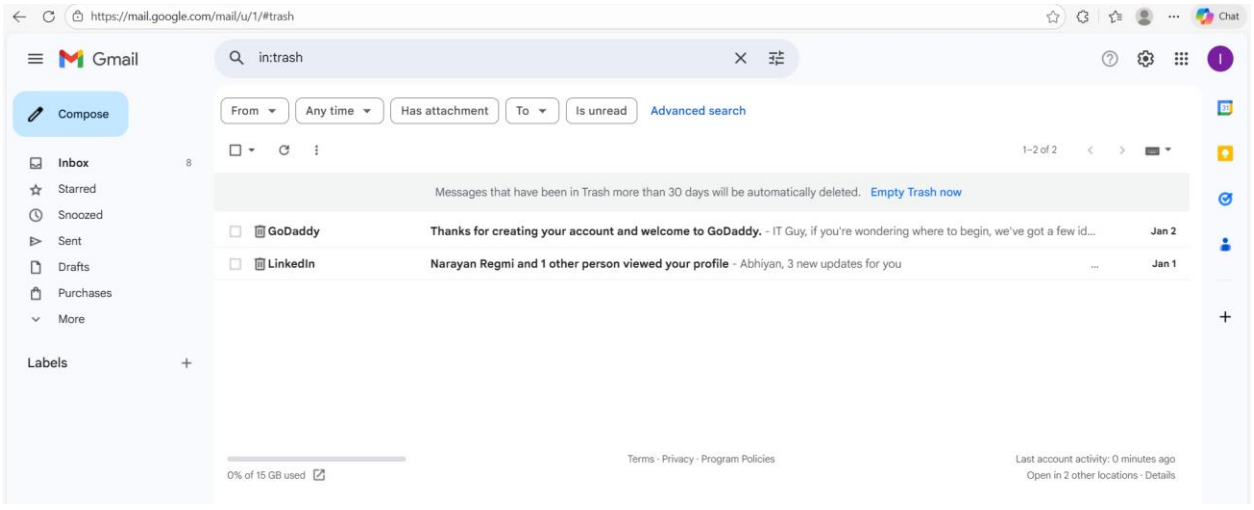


Fig: The system successfully moved the mail that is suspicious to trash

Sample logs.

Fig: System Startup Log

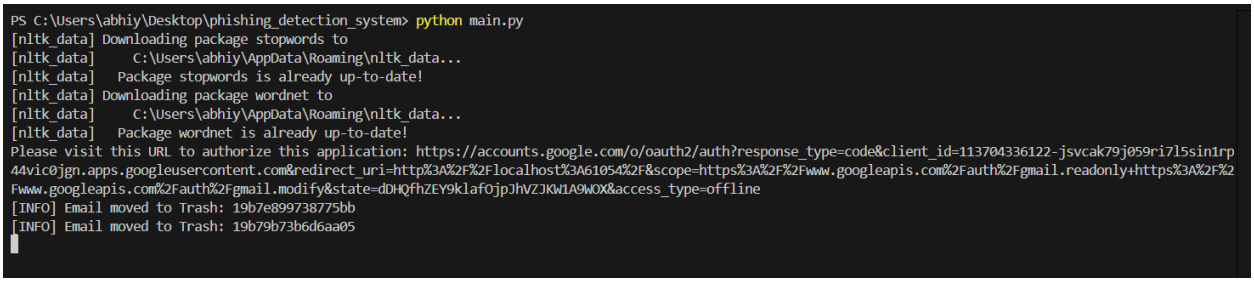


Fig: Phishing email is moved to trash logs

GUI screenshots.

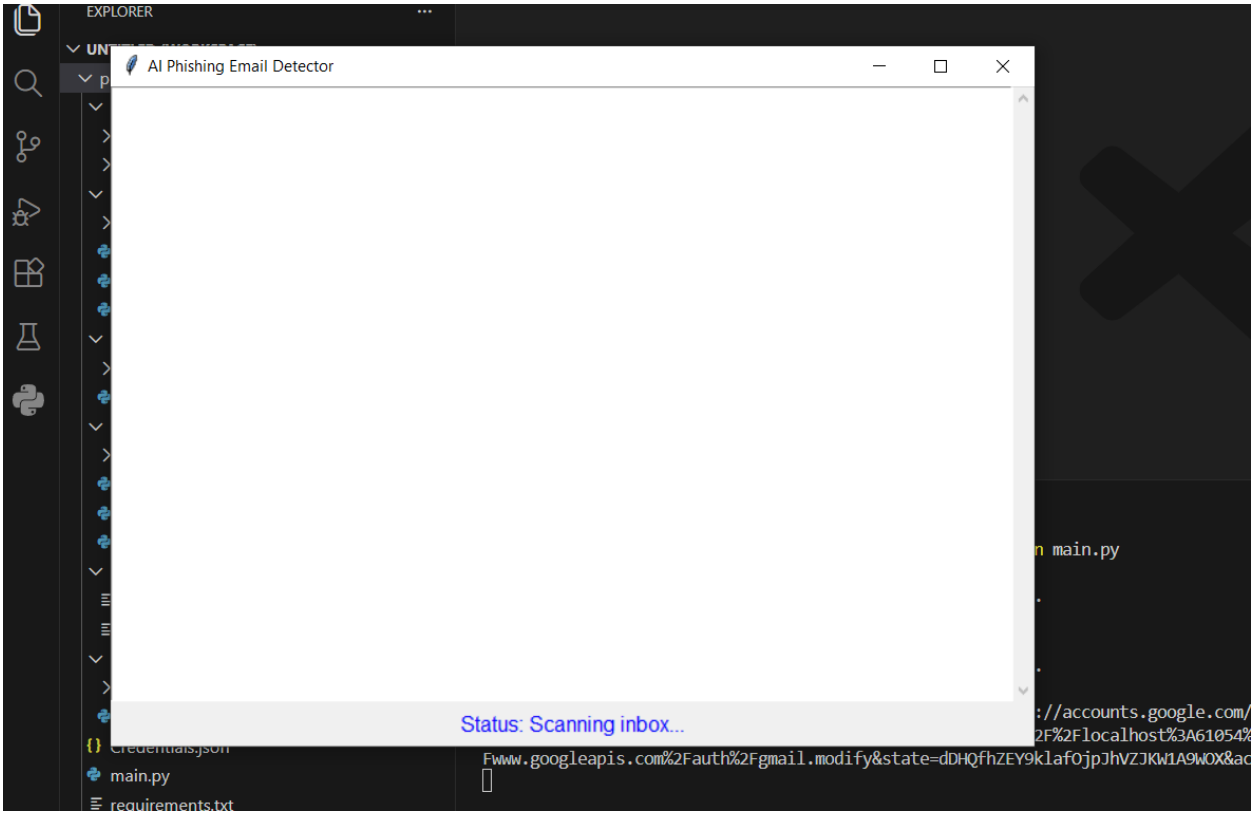


Fig: Scanning the emails that is in inbox

Adversarial Considerations

Potential adversarial tactics include:

- Obfuscated text.
- Image-based phishing.
- Benign-looking language with malicious links.

These represent areas for future enhancement.

Limitations

- Binary classification only.
- No attachment or URL reputation analysis.
- Desktop-only deployment.
- Dependence on dataset quality.

Conclusion and Recommendations

Conclusion

Phishing attacks continue to represent one of the most persistent and damaging threats in modern cybersecurity, primarily due to their reliance on social engineering rather than technical exploitation. As email remains a dominant medium for personal and professional communication, the need for intelligent, adaptive, and user-centric phishing detection mechanisms has become increasingly critical. This project successfully addressed this challenge by designing and implementing an AI-based phishing email detection system integrated with Gmail, combining machine learning, natural language processing, and secure API-based automation.

The developed system demonstrates that classical machine learning techniques, when supported by effective text preprocessing and feature engineering, can achieve reliable phishing detection performance in real-world scenarios. By leveraging TF-IDF feature extraction and a Logistic Regression classifier, the system is able to identify linguistic and contextual patterns commonly associated with phishing emails. The preprocessing pipeline, which includes stopword removal, lemmatization, HTML stripping, and URL normalization, plays a crucial role in reducing noise and improving model generalization. The achieved classification accuracy on the test dataset validates the effectiveness of the chosen approach and confirms the suitability of lightweight models for real-time email security applications.

A key strength of this project lies in its practical integration with Gmail using OAuth 2.0 authentication. Unlike theoretical or offline-only models, this system operates directly on live user emails while maintaining strong security and privacy guarantees. Sensitive credentials are never stored in plaintext, and access is limited to the minimum required API scopes. The automated mitigation strategy, which moves detected phishing emails to the Trash rather than permanently deleting them, reflects a responsible and user-safe design choice that minimizes the impact of false positives. Furthermore, the inclusion of a Tkinter-based graphical user interface enhances usability and transparency by allowing users to monitor scanning activity and detection outcomes in real time.

Despite its success, the system has certain limitations that provide opportunities for future improvement. The current implementation focuses solely on email body text and binary classification, which may limit detection capability against advanced phishing techniques such as image-based phishing, malicious attachments, or context-aware social engineering attacks. Additionally, the reliance on a single public dataset may restrict the model's ability to generalize across diverse email environments. Addressing these limitations would require incorporating additional data sources, multi-modal analysis, and more advanced machine learning architectures.

In conclusion, this project demonstrates the feasibility and effectiveness of an AI-driven approach to phishing email detection in a real-world Gmail environment. It highlights how machine learning and NLP can be practically applied to enhance user security while maintaining ethical standards and system transparency. The modular and extensible design of the system provides a strong foundation for future research, enterprise deployment, and further innovation in AI-based email security solutions.

Ethical Considerations

- User privacy and data minimization.
- Secure handling of OAuth credentials.
- Avoidance of irreversible actions.

Recommendations and Future Work

- Integrate deep learning models (LSTM, BERT).
- Add URL and attachment analysis.
- Deploy as a browser extension or cloud service.
- Improve explainability with feature importance visualization.
- Conduct large-scale user studies.

References and Bibliography

References

.owasp, n.d. *owasp.* [Online]
Available at: https://cheatsheetseries.owasp.org/

Alam, N. A., n.d. *Kaggle.* [Online]
Available at: https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset/

Bishop, C. M., 2006. *archive.org.* [Online]
Available at: https://archive.org/details/patternrecogniti0000bish

Dan Jurafsky, J. H. M., 2025. *stanford.edu.* [Online]
Available at: https://web.stanford.edu/~jurafsky/slp3/

Fette, S. T., 2007. *FetteSadehTomasicWWW2007.pdf.* s.l.:s.n.

Google., n.d. *Google. Gmail API Documentation..* [Online]
Available at: https://developers.google.com/workspace/gmail/api/guides

jain, g., 2018. A machine learning based approach for phishing detection using hyperlinks information. *A machine learning based approach for phishing detection using hyperlinks information,* 26 April.

NIST, n.d. *NIST.* [Online]
Available at: https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

openai, n.d. *openai.* [Online]
Available at: https://openai.com/safety/

scikit-learn, n.d. *scikit-learn.* [Online]
Available at: https://scikit-learn.org/stable/index.html

Appendix

Source code snippets

```python
def clean_text(text):
    """
    Clean email text:
    - Lowercase
    - Remove HTML
    - Replace URLs with 'url'
    - Remove special characters
    - Remove stopwords
    - Lemmatize
    """
    text = str(text).lower()

    # Replace URLs with 'url' token
    urls = re.findall(r"http\S+", text)
    text = re.sub(r"http\S+", " url ", text)

    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"[^a-zA-Z0-9\s]", " ", text)

    tokens = text.split()
    stop_words = set(stopwords.words('english'))
    lemmatizer = WordNetLemmatizer()
    cleaned = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_words]

    if urls:
        cleaned.append("url")

    return " ".join(cleaned)
```

You have Docker insta
install the recommend

Purpose:

Ensures consistent and noise-free textual input for machine learning classification.

```
vectorizer = TfidfVectorizer(max_features=5000)
X_vec = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size=0.2, random_state=42)

model = OneVsRestClassifier(LogisticRegression(solver='liblinear', max_iter=1000))
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

os.makedirs("model", exist_ok=True)
with open("model/phishing_model.pkl", "wb") as f:
    pickle.dump(model, f)
with open("model/vectorizer.pkl", "wb") as f:
    pickle.dump(vectorizer, f)

print("Model and vectorizer saved in 'model/' folder.")
```

Purpose:

Transforms textual data into numerical vectors and trains the phishing detection model.

```
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
        else:
            if not os.path.exists(CREDENTIALS_FILE):
                raise FileNotFoundError("credentials.json not found")
            flow = InstalledAppFlow.from_client_secrets_file(
                CREDENTIALS_FILE, GMAIL_SCOPES
            )
            creds = flow.run_local_server(port=0)

        with open(TOKEN_FILE, "w") as token:
            token.write(creds.to_json())
```

Purpose:

Provides secure, user-consented access to Gmail without storing passwords.

Additional diagrams.

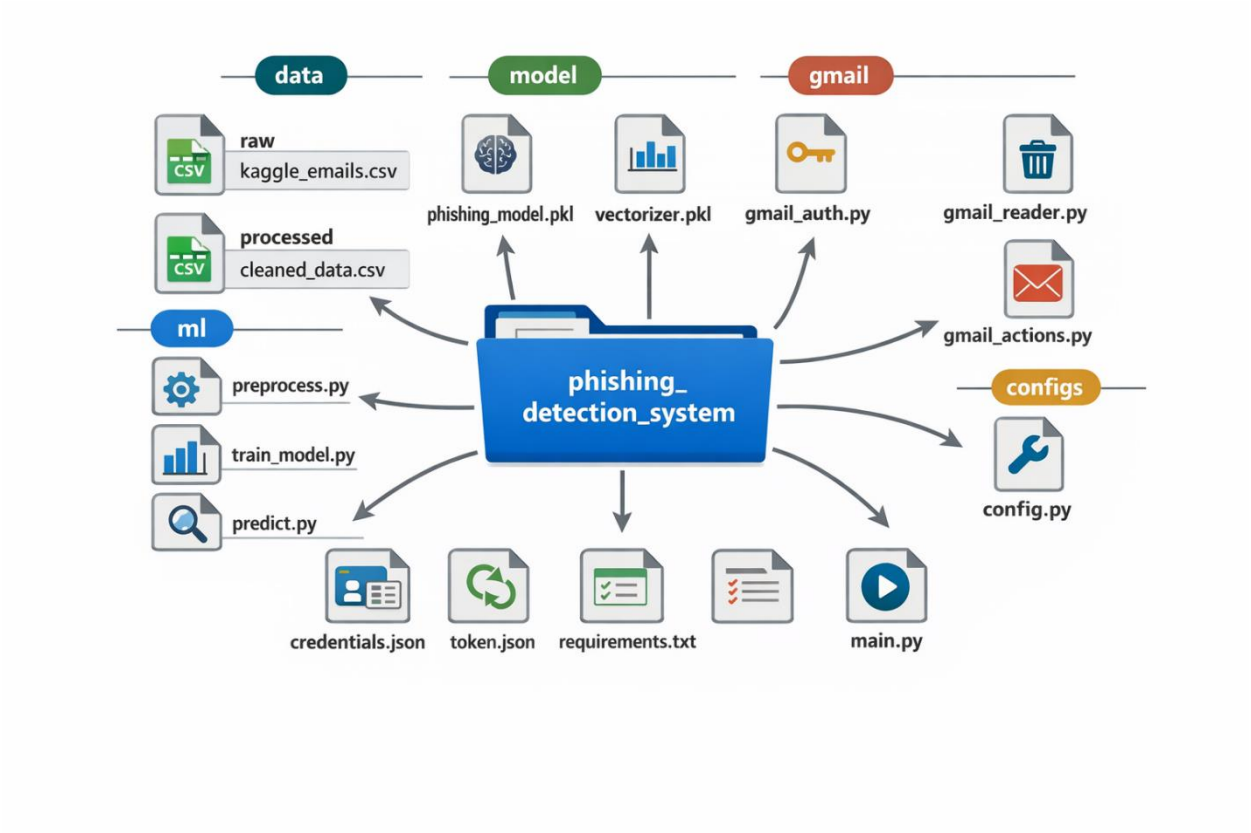


Fig: File Structure

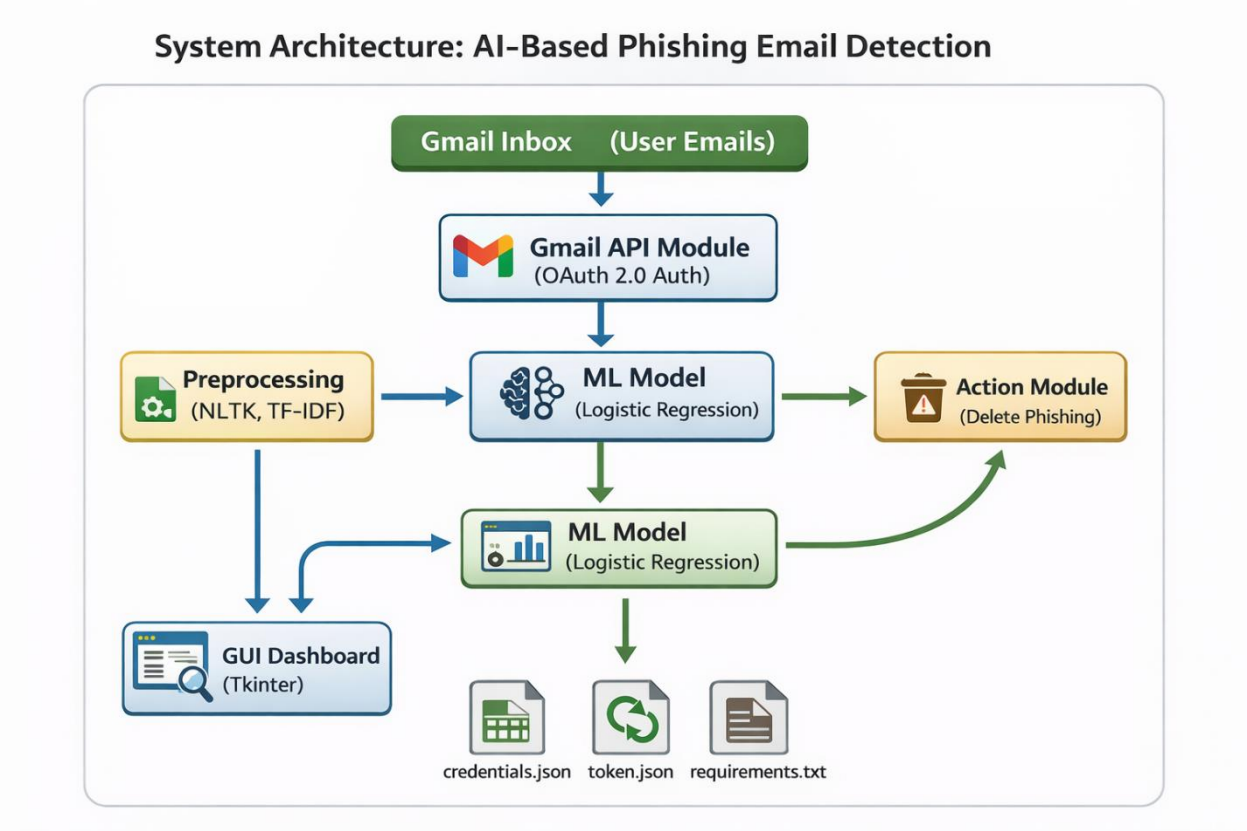


Fig: System Architecture of the System

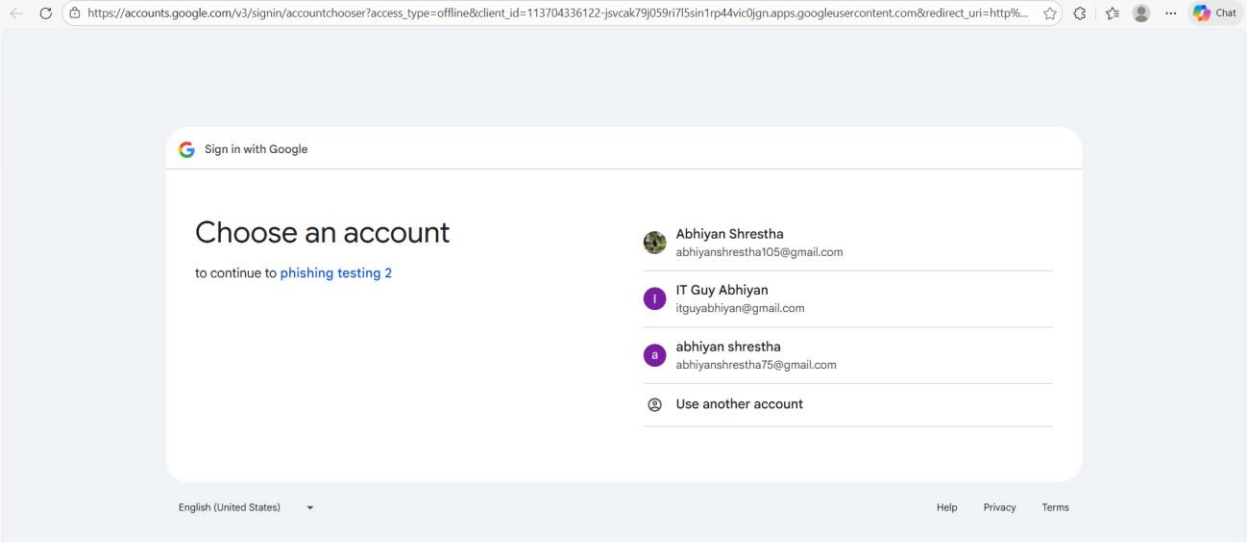


Fig: Choosing the email which i want to connect.

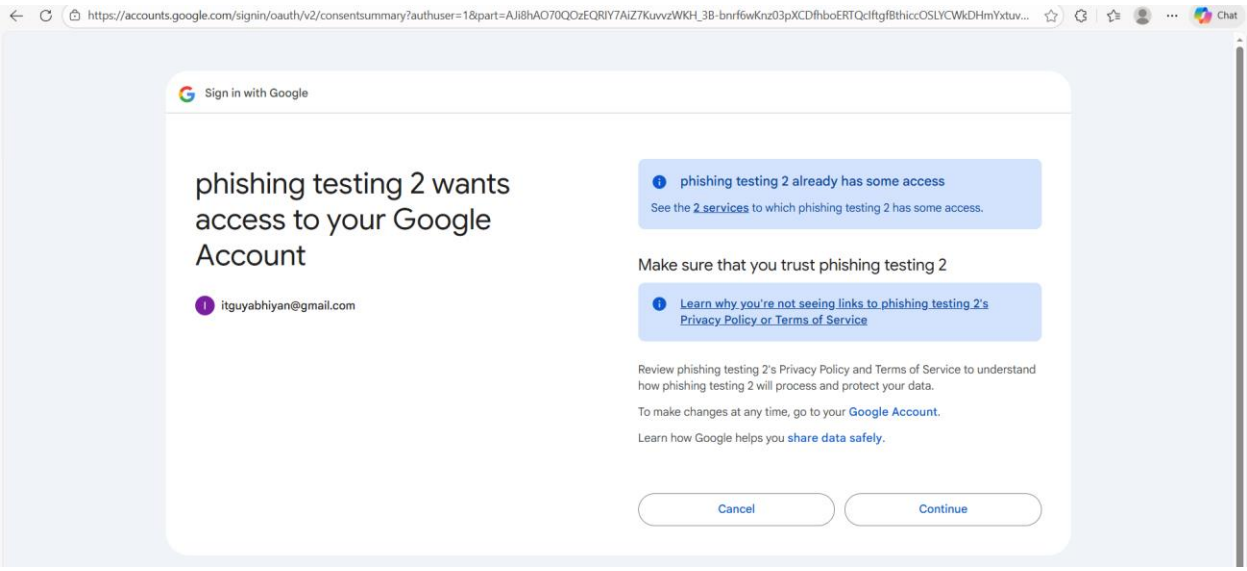


Fig: Asking for permission to access and modify the mail

Fig: Connection is Successful

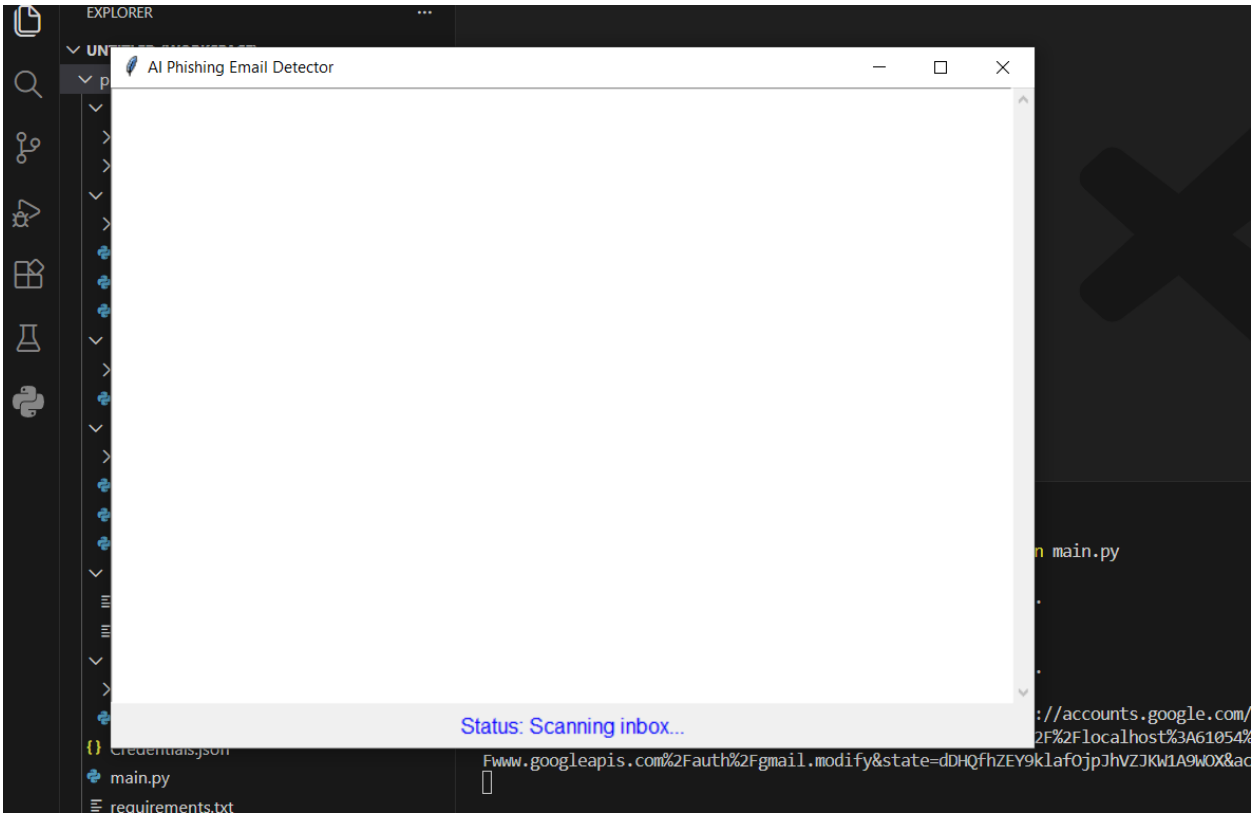


Fig: The System is Scanning emails

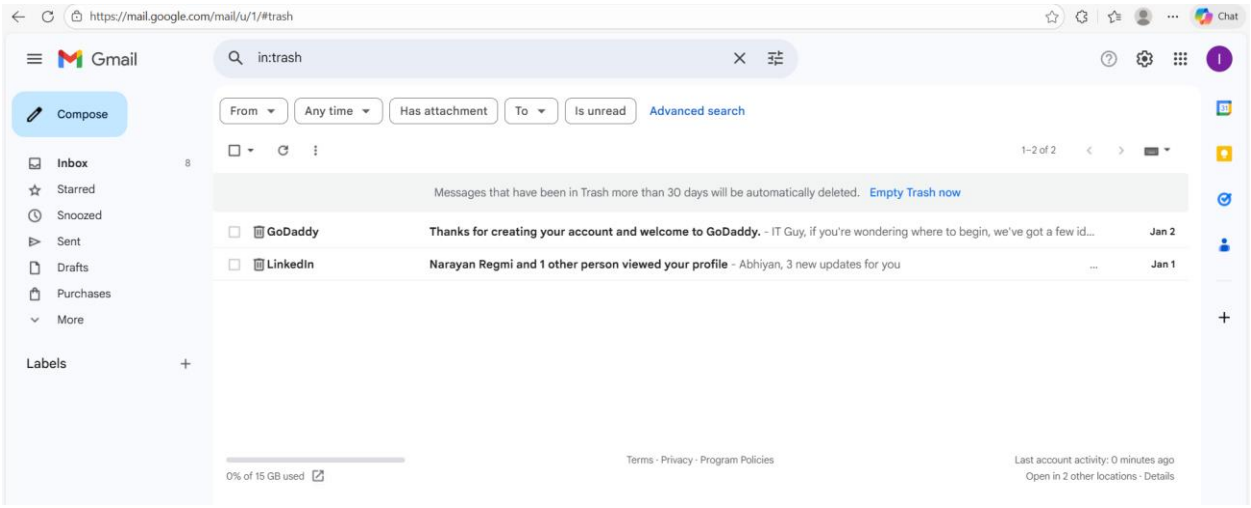


Fig: The system successfully moved the mail that is suspicious to trash

Sample logs.

```
PS C:\Users\abhiy\Desktop\phishing_detection_system> python main.py
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\abhiy\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\abhiy\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Please visit this URL to authorize this application: https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=113704336122-jsvcak79j059ri7l5sin1rp
44vic0jgn.apps.googleusercontent.com&redirect_uri=http%3A%2F%2Flocalhost%3A61054%2F&scope=https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fgmail.readonly+https%3A%2F%2
Fwww.googleapis.com%2Fauth%2Fgmail.modify&state=dDHQfhZEY9klafOjpJhVZJKW1A9WOX&access_type=offline
```

Fig: System Startup Log

```
PS C:\Users\abhiy\Desktop\phishing_detection_system> python main.py
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\abhiy\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\abhiy\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Please visit this URL to authorize this application: https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=113704336122-jsvcak79j059ri7l5sin1rp
44vic0jgn.apps.googleusercontent.com&redirect_uri=http%3A%2F%2Flocalhost%3A61054%2F&scope=https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fgmail.readonly+https%3A%2F%2
Fwww.googleapis.com%2Fauth%2Fgmail.modify&state=dDHQfhZEY9klafOjpJhVZJKW1A9WOX&access_type=offline
[INFO] Email moved to Trash: 19b7e899738775bb
[INFO] Email moved to Trash: 19b79b73b6d6aa05
```

Fig: Phishing email is moved to trash logs

GUI screenshots.

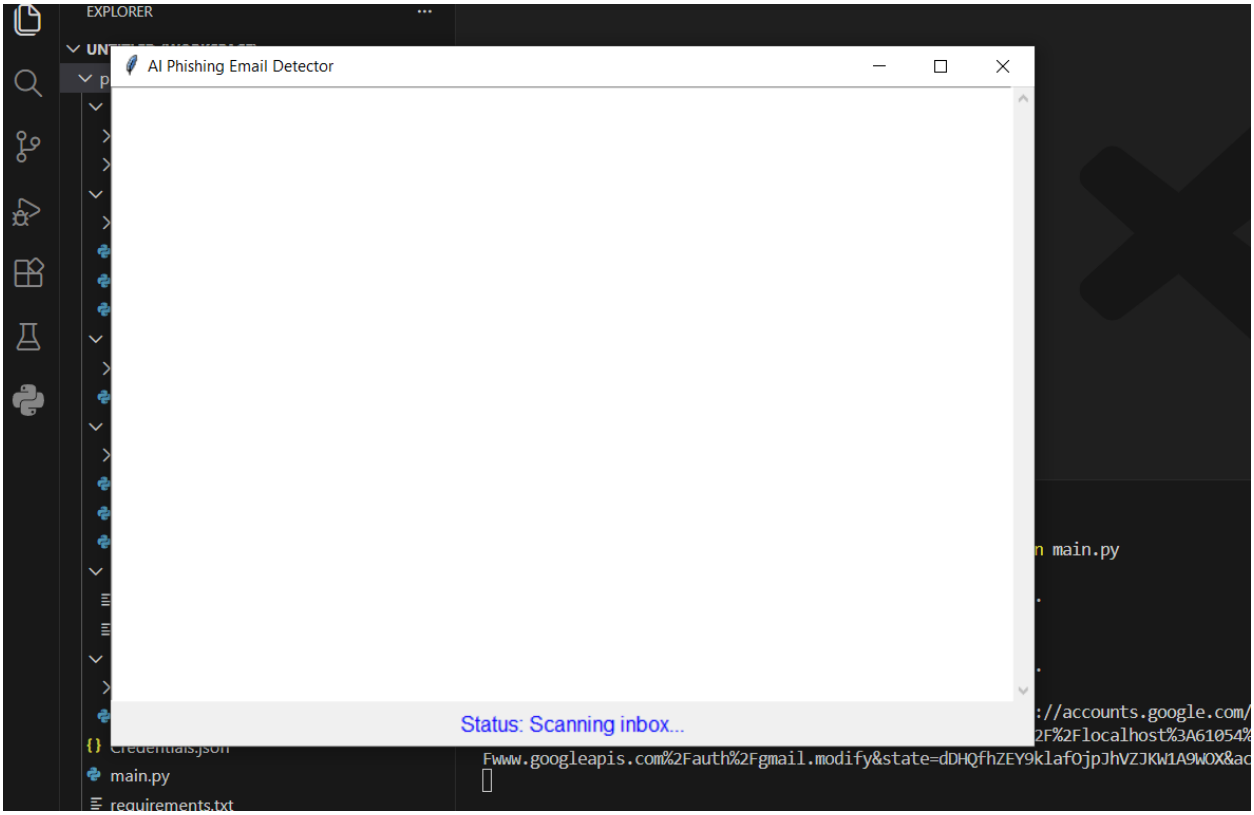
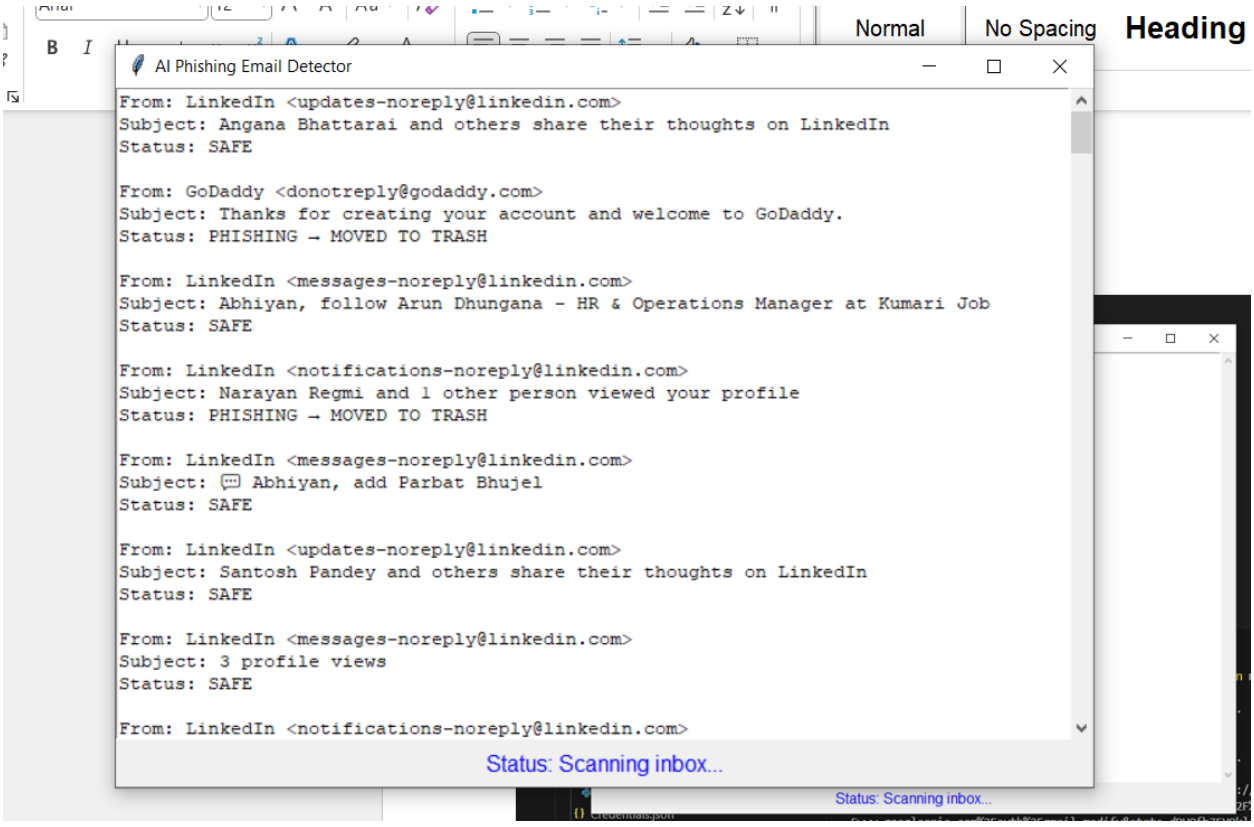


Fig: Scanning the emails that is in inbox



Description:

Displays sender details, email subject, and classification results (Safe or Phishing).